

Historia do c++

Antes de falarmos sobre a linguagem de programação C++, é necessário que entendamos as motivações e limitações de seu predecessor. O C surgiu no início da década de 1970, desenvolvida por Dennis Ritchie, com o objetivo de criar sistemas operacionais eficientes e portáteis. Sua principal motivação foi oferecer controle direto sobre o hardware com desempenho elevado, mantendo uma sintaxe mais estruturada do que o assembly. Por isso, o C se tornou amplamente utilizado em sistemas embarcados, drivers, sistemas operacionais e aplicações onde desempenho e previsibilidade são críticos. No entanto, o C possui **limitações**, principalmente nos quesitos de organização de códigos grandes e proteção do código quanto a usos errados. Todos esses fatores tornam muito difícil a sua utilização e manutenção em sistemas maiores e mais complexos.



Dennis Ritchie, criador da linguagem C.

Por conta dessas limitações, o C++ foi criado por Bjarne Stroustrup como uma **evolução do C**, com o objetivo de manter seu alto desempenho e controle sobre o hardware, mas oferecendo melhores ferramentas para organizar programas grandes e complexos. A principal ideia do C++ foi permitir que o código fosse estruturado em torno de objetos, que representam entidades do mundo real e agrupam dados e comportamentos relacionados, facilitando a organização, reutilização e manutenção do software. Essa abordagem, conhecida como programação orientada a objetos, ajuda a reduzir erros e tornar sistemas extensos mais compreensíveis. Por isso, o C++ é amplamente utilizado em áreas como robótica, jogos, sistemas embarcados, simulações e softwares de alto desempenho.



Bjarne Stroustrup, criador da linguagem C++.

Meu primeiro código

Eu imagino o quão animado você esteja para começar a sua jornada com o C++. mas antes de tudo é necessário entender os componentes básicos para se compor um código nessa linguagem de programação. Vamos analisar o exemplo abaixo e ir destrinchando cada parte. Rode cada um dos blocos abaixo individualmente.

```
In [1]: # Rode o comando abaixo para poder usar os códigos em cpp
!mkdir cpp_files
```

Entendendo cada estrutura do código.

1. `#include <iostream>` : A primeira parte de um programa em C++ é a inclusão de bibliotecas, que fornecem funcionalidades prontas para o código. No exemplo abaixo, a biblioteca `<iostream>` permite realizar operações de entrada e saída, como imprimir mensagens no terminal. Note que essa declaração será substituída na etapa de pré-processamento pelo conteúdo completo da biblioteca.
2. `using namespace std;` : Em C++, muitas funcionalidades estão organizadas dentro de espaços de nomes (namespaces), que ajudam a evitar conflitos entre nomes de variáveis e funções. O comando `using namespace std;` indica que estamos utilizando o este namespace como padrão, permitindo escrever `cout` e `endl` de forma simplificada. Abordaremos mais à frente no curso como criar seus próprios namespaces.
3. `int main(){}` : Todo programa em C++ começa sua execução a partir da função `main`. Ela define o ponto de entrada do programa e contém as instruções que serão executadas quando o programa for iniciado.

4. `cout<<"Olá, Sonho!"<<endl;` : Dentro da função `main` , escrevemos as instruções que compõem o comportamento do programa. No exemplo, utilizamos o comando `cout` para imprimir uma mensagem no terminal.
5. `//eu sou um comentario` : Comentários em C++ são feitos a partir de duas barras e acabam na mesma linha. Caso queira um comentario que percorra mais de uma linha, você pode usar `/*` no início e completar com `*/` o seu comentário.
6. `return 0` : A instrução `return 0`; indica que o programa foi finalizado com sucesso. Esse valor é retornado ao sistema operacional e, por convenção, o valor zero representa uma execução sem erros.

```
In [8]: %%writefile cpp_files/primeiro_exemplo.cpp
#include <iostream>
using namespace std;

int main(){

    cout<<"Olá, Sonho!"<<endl;

    //eu sou um comentario

    return 0;
}
```

Overwriting cpp_files/primeiro_exemplo.cpp

```
In [ ]: !g++ cpp_files/primeiro_exemplo.cpp -o cpp_files/primeiro_exemplo
```

Linguagem compilada vs interpretada

Durante o curso do `Sonho` , os alunos (você) terão contato com dois tipos de linguagens de programação: Linguagens **Compiladas** e **Interpretadas**, sendo elas representadas neste curso pelo `C++` e pelo `Python` , respectivamente.

Linguagens interpretadas

Nas linguagens interpretadas, como o Python, o código não é transformado previamente em um programa executável. Em vez disso, um interpretador lê e executa o código diretamente, traduzindo cada instrução durante a execução do programa. Isso facilita testes rápidos, experimentação e prototipagem, tornando o desenvolvimento mais ágil e acessível, especialmente nas fases iniciais de um projeto. Em contrapartida, essa tradução em tempo de execução geralmente resulta em menor desempenho e menor controle sobre os recursos do sistema, o que pode ser um fator limitante em aplicações críticas ou de tempo real. Por esse motivo, linguagens interpretadas são muito utilizadas em automação, scripts, análise de dados e interfaces, enquanto linguagens compiladas dominam áreas onde desempenho e confiabilidade são essenciais.

Ao longo do curso, ficará claro que essas duas abordagens não competem entre si, mas se complementam. Em robótica, é comum utilizar linguagens interpretadas para testes e desenvolvimento rápido, ou para tarefas onde a eficiência não é essencial, enquanto linguagens compiladas são empregadas nas partes do sistema que exigem maior desempenho, estabilidade e controle.

Linguagens compiladas

Em linguagens compiladas, como o C++, o código escrito pelo programador passa por um processo de compilação, no qual o compilador traduz o código de uma linguagem compreensível por humanos para linguagem de máquina, que pode ser executada diretamente pelo processador.

Esse processo envolve etapas como verificação de erros, otimizações e geração de um arquivo executável. Como resultado, o programa final tende a ser mais rápido, eficiente e previsível, características fundamentais em sistemas embarcados, robótica e aplicações que exigem controle preciso do tempo de execução. A principal desvantagem das linguagens compiladas é que o ciclo de desenvolvimento costuma ser mais rígido: qualquer erro exige uma nova compilação antes que o programa possa ser executado novamente. Isso demanda maior atenção à escrita do código e uma compreensão mais clara do funcionamento do sistema desde o início.

Processo de compilação no C++

Em C++, a compilação não ocorre em um único passo. Abaixo estão listadas as principais etapas no processo de compilação de um código em C++.

1. Pré-processamento

- Lida com as linhas que começam com `#`, como `#include` e `#define`, preparando o código para o compilador.
- Substitui a declaração `#include` pelo código das bibliotecas que foram incluídas.
- Expande macros, substituindo declarações de `define` pelo seu valor real.
 - *Exemplo: Em um código onde se tenha um `#define PI 3.14`, onde no código aparecer `PI`, será substituído por `3.14`.*
- Lida com compilações condicionais, como `#ifdef`.
- Esse processo cria um arquivo `.i`

2. Compilação

- Essa etapa é responsável pela tradução do código C++ para a linguagem Assembly.
- Realiza a checagem de sintaxe do código.
- Esse processo cria um arquivo `.s`

3. Montagem

- Etapa responsável pela conversão do código Assembly em "código objeto", linguagem de máquina, mas ainda não executável.

- Esse processo cria um arquivo `.o`

4. Linkagem

- Cria um arquivo executável.

É importante saber que todas essas etapas ocorrem, comumente, de maneira interna ao comando de compilação que se é usado. A compilação dos códigos será explorada mais a frente nessa aula, onde trabalharemos a geração dos arquivos intermediários, assim como a compilação mais "padrão".

Principais compiladores do C++

Para realizar o processo de compilação, é necessário utilizar, não surpreendentemente, um compilador. Ele é o responsável por toda a pipeline de compilação que apresentamos anteriormente. No ambiente Linux, especialmente no Ubuntu, o compilador mais comum é o GCC (GNU Compiler Collection). Apesar do nome, o GCC não é apenas um compilador exclusivo de C, possuindo uma coleção de compiladores para várias linguagens, incluindo C, C++, entre outras. Quando trabalhamos com C++, geralmente usamos o comando `g++`, que é a interface do GCC específica para C++.

Embora ambos façam parte do mesmo conjunto de ferramentas, existe uma diferença importante: ao usar `g++`, o sistema automaticamente inclui as bibliotecas padrão do C++. Se você tentar compilar um programa com `gcc` usando apenas `gcc`, provavelmente terá erros de linkagem, pois a biblioteca padrão do C++ não será vinculada automaticamente. Por isso, ao longo do curso, **utilizaremos sempre `g++` para compilar nossos programas em C++**.

Além do `g++`, existe outro compilador bastante utilizado chamado Clang, que faz parte do projeto LLVM. O Clang é conhecido por gerar mensagens de erro mais claras e detalhadas, o que pode ajudar bastante durante o aprendizado. Em muitos sistemas macOS, por exemplo, o Clang é o compilador padrão. Na prática, tanto o `g++` quanto o Clang produzem executáveis equivalentes e seguem os padrões da linguagem C++. A escolha entre eles costuma depender do sistema operacional, das ferramentas do projeto ou da preferência da equipe.

```
In [10]: !echo "1) Gerando arquivo pré-processado (.i)..."
!g++ -E cpp_files/primeiro_exemplo.cpp -o cpp_files/primeiro_exemplo.i

!echo "2) Gerando código em Assembly (.s)..."
!g++ -S cpp_files/primeiro_exemplo.i -o cpp_files/primeiro_exemplo.s

!echo "3) Gerando arquivo objeto (.o) - código de máquina, ainda não executável"
!g++ -c cpp_files/primeiro_exemplo.s -o cpp_files/primeiro_exemplo.o

!echo "4) Linkando (juntando) o .o com bibliotecas para gerar o executável final"
!g++ cpp_files/primeiro_exemplo.o -o cpp_files/primeiro_exemplo
```

```
!echo "5) Executando o código..."  
!./cpp_files/primeiro_exemplo
```

- 1) Gerando arquivo pré-processado (.i)...
 - 2) Gerando código em Assembly (.s)...
 - 3) Gerando arquivo objeto (.o) - código de máquina, ainda não executável...
 - 4) Linkando (juntando) o .o com bibliotecas para gerar o executável final...
 - 5) Executando o código...
- Olá, Sonho!